

Módulo 6 · Arrays y colecciones

Guardar muchos datos, recorrerlos, buscar información y elegir la estructura adecuada.

De una variable suelta a una colección de datos

Objetivo: que el alumnado deje de crear variable1, variable2, variable3... y aprenda a trabajar con listas de elementos.

- Arrays para tamaños fijos
- ArrayList para listas que crecen o disminuyen
- Map para asociar claves con valores
- Bucles para recorrer y procesar datos

Resultados de aprendizaje

Qué debería saber hacer el alumnado al terminar el módulo.

- Declarar arrays y comprender que empiezan en la posición 0.
- Recorrer arrays con for clásico y for-each.
- Calcular media, mínimo, máximo y contadores.
- Usar ArrayList para añadir, mostrar y eliminar datos.
- Entender Map como asociación clave-valor.

Idea docente: no basta con que el código funcione; deben poder explicar qué contiene cada estructura y cómo se recorre.

¿Por qué necesitamos arrays?

Cuando tenemos varios datos del mismo tipo, una sola variable ya no alcanza.

Sin arrays

- `String alumno1 = "Ana"`
- `String alumno2 = "Luis"`
- `String alumno3 = "Marta"`
- Difícil de recorrer
- Difícil de escalar

Con arrays

- `String[] alumnos = {...}`
- Todos los nombres en una estructura
- Podemos usar bucles
- El código se simplifica
- Más fácil calcular estadísticas

Qué es un array

Un array es una estructura que guarda varios valores del mismo tipo.

```
int[] notas = {7, 5, 9, 4};
```

- El tipo `int[]` indica que guardamos varios enteros.
- Cada valor ocupa una posición dentro del array.
- El tamaño se fija al crearlo.
- Todos los elementos tienen el mismo tipo.

Analogía útil: un array es como una fila de taquillas numeradas. Cada taquilla guarda un dato.

Posiciones: el índice empieza en 0

Este punto suele provocar muchos errores al empezar.

```
int[] notas = {7, 5, 9, 4};
```

```
System.out.println(notas[0]); // 7
```

```
System.out.println(notas[1]); // 5
```

```
System.out.println(notas[3]); // 4
```

- La primera posición es 0, no 1.
- La última posición es `length - 1`.
- En este ejemplo hay 4 notas: posiciones 0, 1, 2 y 3.
- Intentar acceder a `notas[4]` daría error.

Crear arrays de dos formas

Podemos crear el array con valores iniciales o reservar tamaño.

```
// Array con valores iniciales
int[] notas = {7, 5, 9, 4};

// Array con tamaño fijo
String[] nombres = new String[3];
nombres[0] = "Ana";
nombres[1] = "Luis";
nombres[2] = "Marta";
```

- La primera forma es más cómoda cuando ya conocemos los datos.
- La segunda se usa cuando sabemos cuántos elementos habrá, pero aún no conocemos sus valores.
- En ambos casos el tamaño queda fijado.

Recorrer un array con for clásico

Útil cuando necesitamos conocer la posición de cada elemento.

```
int[] notas = {7, 5, 9, 4};

for (int i = 0; i < notas.length; i++) {
    System.out.println("Posición " + i + ": " + notas[i]);
}
```

- i empieza en 0.
- La condición `i < notas.length` evita salirnos del array.
- `i++` avanza una posición en cada vuelta.
- `notas[i]` accede al valor de la posición actual.

Recorrer un array con for-each

Más limpio cuando no necesitamos la posición.

```
int[] notas = {7, 5, 9, 4};  
  
for (int nota : notas) {  
    System.out.println(nota);  
}
```

- Se lee como: “para cada nota dentro de notas”.
- Reduce errores con índices.
- Es ideal para mostrar o sumar valores.
- No sirve si necesitamos modificar una posición concreta.

Acumuladores: sumar valores

Un acumulador guarda un resultado que va creciendo dentro del bucle.

```
int[] notas = {7, 5, 9, 4};  
int suma = 0;  
  
for (int nota : notas) {  
    suma += nota;  
}  
  
System.out.println("Suma: " + suma);
```

suma += nota equivale a suma = suma + nota.
En cada vuelta se añade el valor actual.

- Antes del bucle: suma vale 0.
- Durante el bucle: se añaden las notas.
- Después del bucle: tenemos la suma total.

Calcular una media

La media se calcula dividiendo la suma entre el número de elementos.

```
int[] notas = {7, 5, 9, 4};
int suma = 0;

for (int nota : notas) {
    suma += nota;
}

double media = (double) suma / notas.length;
System.out.println("Media: " + media);
```

- `notas.length` devuelve cuántos elementos tiene el array.
- `(double)` fuerza una división decimal.
- Sin esa conversión podríamos perder decimales.

Buscar mínimo y máximo

Para encontrar extremos se parte de un valor inicial y se compara.

```
int[] notas = {7, 5, 9, 4};
int minimo = notas[0];
int maximo = notas[0];

for (int nota : notas) {
    if (nota < minimo) minimo = nota;
    if (nota > maximo) maximo = nota;
}

System.out.println("Mínimo: " + minimo);
System.out.println("Máximo: " + maximo);
```

- No empezamos con 0 porque puede falsear el resultado.
- Usamos el primer elemento como referencia inicial.
- Cada vuelta compara y actualiza si encuentra un valor mejor.

Contadores: contar aprobados

Un contador aumenta cuando se cumple una condición.

```
int[] notas = {7, 5, 9, 4};
int aprobados = 0;

for (int nota : notas) {
    if (nota >= 5) {
        aprobados++;
    }
}

System.out.println("Aprobados: " + aprobados);
```

aprobados++ significa: aumenta el contador en 1.

- Combina arrays, bucles y condicionales.
- Es un patrón muy común en programación.
- Sirve para contar aprobados, productos agotados, tareas pendientes, etc.

Error frecuente: salir del array

`IndexOutOfBoundsException` aparece al acceder a una posición inexistente.

```
int[] notas = {7, 5, 9, 4};  
  
System.out.println(notas[4]); // ERROR  
  
// Última posición correcta:  
System.out.println(notas[notas.length - 1]);
```

- El array tiene 4 elementos, pero sus índices son 0, 1, 2 y 3.
- `notas[4]` intenta entrar en una quinta posición.
- La última posición siempre es `length - 1`.

Limitación del array

Los arrays son útiles, pero su tamaño no cambia una vez creados.

Ventajas

- Rápidos y sencillos
- Muy útiles para datos fijos
- Buena base para aprender bucles
- El acceso por índice es directo

Limitaciones

- Tamaño fijo
- Añadir o eliminar elementos es incómodo
- Hay que controlar bien los índices
- Para listas dinámicas conviene ArrayList

ArrayList: una lista flexible

ArrayList permite añadir y eliminar elementos con más comodidad.

```
import java.util.ArrayList;
import java.util.List;

List<String> tareas = new ArrayList<>();
tareas.add("Estudiar Java");
tareas.add("Hacer ejercicios");
tareas.remove("Estudiar Java");
```

- No indicamos tamaño fijo.
- add() añade elementos.
- remove() elimina elementos.
- Es más cómodo para datos que cambian.

Recorrer un ArrayList

Se puede recorrer de forma muy parecida a un array.

```
List<String> nombres = new ArrayList<>();
nombres.add("Ana");
nombres.add("Luis");
nombres.add("Marta");

for (String nombre : nombres) {
    System.out.println(nombre);
}
```

En ArrayList usamos size() para saber cuántos elementos hay. En arrays usamos length.

```
nombres.size();    // ArrayList
notas.length;      // array
```

Operaciones básicas de ArrayList

Las listas tienen métodos preparados para trabajar con sus elementos.

```
nombres.add("Ana");           // añadir
nombres.get(0);               // leer por posición
nombres.set(0, "Eva");        // modificar
nombres.remove(0);            // eliminar por posición
nombres.size();               // tamaño
nombres.contains("Luis");     // buscar
```

- Los métodos terminan con paréntesis porque son acciones.
- `get(0)` lee el primer elemento.
- `set()` cambia un valor existente.
- `contains()` devuelve true o false.

Array vs ArrayList

Elegir una estructura u otra depende del problema.

Usar array cuando...

- El tamaño está claro desde el principio
- Trabajamos con datos fijos
- Queremos practicar índices y bucles
- Ejemplo: notas de un examen cerrado

Usar ArrayList cuando...

- La cantidad puede cambiar
- Necesitamos añadir o eliminar
- Queremos un menú de tareas
- Ejemplo: lista de alumnos inscritos

Map: clave y valor

Un Map guarda datos asociados a una clave única.

```
import java.util.HashMap;  
import java.util.Map;  
  
Map<String, Double> precios = new HashMap<>();  
precios.put("Pan", 1.20);  
precios.put("Leche", 1.05);  
  
System.out.println(precios.get("Pan"));
```

- La clave es String: "Pan", "Leche".
- El valor es Double: 1.20, 1.05.
- put() guarda o actualiza.
- get() recupera el valor de una clave.

Cuándo usar Map

Map es ideal cuando queremos buscar por nombre, código o identificador.

- Código de producto → precio
- DNI → nombre de persona
- Email → usuario
- Palabra → número de apariciones
- Producto → stock disponible

Diferencia clave: en un array o ArrayList se busca por posición; en un Map se busca por clave.

```
precios.get("Pan");  
stock.get("A001");  
usuarios.get("ana@email.com");
```

Recorrer un Map

Para recorrer un Map se suele trabajar con sus claves o sus entradas.

```
Map<String, Double> precios = new HashMap<>();
precios.put("Pan", 1.20);
precios.put("Leche", 1.05);

for (String producto : precios.keySet()) {
    System.out.println(producto + ": " + precios.get(producto));
}
```

- `keySet()` devuelve las claves.
- Con cada clave podemos obtener su valor con `get()`.
- Es útil para mostrar inventarios, tarifas o diccionarios.

Ejercicio guiado 1: estadísticas de notas

Aplicamos array, bucle, acumulador, mínimo, máximo y contador.

```
/*  
Enunciado:  
Con un array de notas, calcula media, mínimo,  
máximo  
y número de aprobados.  
*/  
  
int[] notas = {7, 5, 9, 4, 6};
```

- Paso 1: crear variables suma, minimo, maximo y aprobados.
- Paso 2: recorrer el array con for-each.
- Paso 3: actualizar acumulador y contadores.
- Paso 4: calcular y mostrar la media.

Ejercicio guiado 2: lista de tareas

Aplicamos ArrayList para añadir, mostrar y eliminar datos.

```
/*  
Enunciado:  
Crea una lista de tareas. Añade tres tareas, muestra la  
lista,  
elimina una tarea y vuelve a mostrarla.  
*/  
  
List<String> tareas = new ArrayList<>();
```

- Usar add() para añadir tareas.
- Usar remove() para eliminar una tarea.
- Usar for-each para mostrar la lista.
- Comprobar el tamaño con size().

Ejercicio guiado 3: precios con Map

Aplicamos clave-valor para guardar precios de productos.

```
/*  
Enunciado:  
Crea un Map con tres productos y sus precios.  
Muestra el precio de un producto concreto.  
*/  
  
Map<String, Double> precios = new HashMap<>();
```

- La clave será el nombre del producto.
- El valor será el precio.
- Usaremos put() para guardar.
- Usaremos get() para consultar.

Errores frecuentes

Qué revisar cuando el programa no funciona.

- Confundir `length` con `size()`: arrays usan `length`, `ArrayList` usa `size()`.
- Acceder a una posición inexistente: `lista.get(10)` cuando solo hay 3 elementos.
- Olvidar imports: `java.util.ArrayList`, `java.util.List`, `java.util.Map`.
- Modificar una lista mientras se recorre sin cuidado.
- Usar un array cuando el tamaño debería cambiar.

Consejo: antes de corregir estilo, comprueba la lógica. ¿Qué dato guardo? ¿Dónde está? ¿Cómo lo recorro?

Buenas prácticas

Pequeñas decisiones que hacen el código más claro.

- Usa nombres descriptivos: notas, tareas, precios, alumnos.
- No mezcles demasiadas responsabilidades en un mismo bloque.
- Inicializa acumuladores y contadores antes del bucle.
- Comprueba los límites: primera y última posición.
- Comenta la intención del código, no cada palabra obvia.

```
int suma = 0;           // acumulador  
int aprobados = 0;      // contador
```

Cierre del módulo

Antes de avanzar a POO, estas ideas deben quedar asentadas.

- Array: guarda muchos valores del mismo tipo con tamaño fijo.
- ArrayList: lista flexible para añadir y eliminar.
- Map: estructura de clave y valor.
- Bucle: herramienta para recorrer estructuras.
- Acumulador y contador: patrones básicos de cálculo.

Próximo paso: unir colecciones con clases y objetos para modelar productos, cuentas, pedidos o inventarios.